

Assignment 3 Code Analysis:

- A. Used Google Gemini and Elon Musk's Grok
- B. I accessed Gemini through Google search (with SearchAI) and I went to the official Grok website to use the model.
- C. Prompt: I want you to create a C++ file program that uses objects NOT FUNCTIONS. This program will prioritize emails for a busy company CEO. You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status. You must implement a MaxHeap using a list-based implementation. Then use MaxHeap to handle all your email prioritizing for the CEO. You must create functions from scratch. Do not include pre-existing heap modules. The test file will begin with the emails in the CEO's Inbox, followed by the commands the CEO will use to read her Inbox. Assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).
Commands are Email <sender category>, <subject line>, <date> which are meant to rank whoever is sending the email in the heap as follows:
Boss - Read first
Subordinate - Read next
Peer - Read next
ImportantPerson - Read next
OtherPerson - Read last
If there's more than one email from the sender, the newest email (not oldest) should be read first. EMAIL is followed by space, other fields are delimited, also assume sender category is one of the 5 senders listed above, assume <subject line> is a string that contains spaces (no commas), and assume <date> is in the format: MM-DD-YYYY
NEXT gives the email for the CEO to read, which displays the information on terminal in following format:
"Sender:
Subject:
Date: "

READ has the CEO reading the email and removing it from the heap and COUNT displays the current count of unread emails. This is what input/output looks like:

- **Sample Test File:**

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

- **Your program should display this:**

```
There are 6 emails to read.

Next email:
    Sender: Boss
    Subject: Never Mind
    Date: 01-03-2025

Next email:
    Sender: Boss
    Subject: Important
    Date: 12-20-2024

There are 4 emails to read.
```

D. Gemini Code:

https://github.com/Somatron/EECS-348-Assignment-3/blob/main/s405j542_Javia_Assignment_3/GeminiEmailPriority.cpp

Grok Code:

https://github.com/Somatron/EECS-348-Assignment-3/blob/main/s405j542_Javia_Assignment_3/GrokEmailPriority.cpp

- E. Both code correctness is distinctively high. Time complexity for BOTH Gemini and Grok is between $O(\log(n))$ and $O(1)$, where NEXT and COUNT are $O(1)$ times while EMAIL and READ are $O(\log(n))$ because it required reorganizing and shuffling email priority queues. Space complexity is also the same, with it being $O(n)$ where as the heap gets

bigger, it grows in size. Maintainability, Groks code is more better documented and formatted than Gemini but both of them have responsible use for handling libraries and initializing variable types from the library class.

- F. I'm choosing Grok because it's easier maintainability (more comments means it understands what it's doing), more organized and simplistic code format and also good file trimming.
- G. Since time and space complexity are nearly flawless, I decided to add a user input terminal instead of having to type something like "cat sample_input.txt | ./GrokElon" everytime you run the file, which allows the user to type out whatever file they want instead of dealing with some linux commands of concatenating text files into the program.